



1

Chapitre 5: PHP



Dorra DHAOU

Maître assistante en Informatique

Docteur et Ingénieur en Informatique

dorra.dhaou@fst.utm.tn / dorradhaou@gmail.com

AU. 2025 - 2026

1

Objectifs

2

- Comprendre le rôle du **langage PHP** dans le développement web
- Créer des pages web **dynamiques**
- Manipuler des **variables, tableaux et fonctions**
- Gérer les **formulaires HTML avec PHP**
- Interagir avec une **base de données (MySQL)**
- Comprendre les notions de :
 - ▣ Sessions
 - ▣ Cookies
 - ▣ Sécurité de base (validation, injection SQL)

2

Plan

3

1. Introduction au PHP (*Concept, et installation*).
2. Bases du langage (*Syntaxe, variables, types, opérateurs*).
3. Structures de contrôle (*Conditions et boucles*).
4. Les Tableaux (*Indexés et associatifs*).
5. Les Fonctions (*Déclaration, paramètres, valeur de retour*).
6. Gestion des formulaires (*Méthodes GET et POST*).
7. Sessions et Cookies (*Gestion de l'état*).
8. Accès aux bases de données (*Connexion avec PDO / MySQLi, CRUD*).

3

Introduction

4

- PHP (**H**ypertext **P**reprocessor) est un langage de programmation **côté serveur** utilisé pour créer des sites web dynamiques.
 - Contrairement à JavaScript (qui s'exécute dans le navigateur), PHP s'exécute **sur le serveur**.
 - Le serveur traite le code et renvoie uniquement du HTML pur au navigateur.
 - **PHP = Générer du HTML dynamiquement**

Pourquoi PHP ?

- Il est open-source et gratuit.
- Il alimente près de 75 % du Web (dont WordPress, Facebook à ses débuts, Wikipedia).
- Il est particulièrement simple à coupler avec des bases de données.

4

Introduction

5

□ Installation de l'environnement de travail :

Comme le PHP est un langage **côté serveur**, le navigateur ne peut pas lire un fichier .php directement depuis votre disque dur. Il nous faut recréer un "serveur local" sur nos machines.

1. Le choix du serveur local (Stack WAMP/XAMPP)

Pour simplifier l'installation, on utilise des packages "tout-en-un" qui installent trois composants essentiels : **Apache, PHP, et MySQL**.

➤ XAMPP :

- Multiplateforme (Windows, Linux, Mac),
- Facile à installer,
- Le plus utilisé pour les débutants, et très complet.

➤ WAMP :

- Fonctionne uniquement sur Windows,
- Très populaire,
- Interface simple et facile à configurer.

5

Introduction

6

□ Étapes d'installation (XAMPP) :

1. Télécharger XAMPP depuis le site officiel
 2. Lancer l'installation
 3. Ouvrir le panneau de contrôle XAMPP
 4. Démarrer : Apache et MySQL
- Pour tester si ça fonctionne, tapez dans le navigateur : <http://localhost> et si tout est correct, la page XAMPP s'affiche.

□ Les fichiers PHP doivent être placés dans le dossier : **htdocs**, par exemple : C:\xampp\htdocs\monprojet\

□ Dans le navigateur : <http://localhost/monprojet/test.php>

```
<?php
echo "PHP fonctionne correctement !";
?>
```

← ↻ ⓘ localhost/monprojet/test.php
PHP fonctionne correctement !

6

7

Bases du langage

Syntaxe, variables, types, opérateurs

7

Premiers pas en PHP

8

- ▣ **Structure d'un script PHP** : Tout code PHP doit être écrit entre **<?php** et **?>**

```
<?php
// code PHP ici
?>
```

- ▣ **Les commentaires**

```
<?php
// commentaire sur une ligne

/*
commentaire
sur plusieurs lignes
*/
?>
```

- ▣ **Affichage**

Pour afficher du texte ou du HTML, on utilise souvent l'instruction **echo**.

```
<?php
echo "Bonjour";
print "Hello";
?>
```

8

Les variables

9

□ Déclaration des variables

En PHP :

- Toutes les variables commencent par \$
- Elles sont sensibles à la casse.
- Pas besoin de déclarer le type (langage à typage faible).

```
<?php
$nom = "Ali";
$age = 20;
?>
```

□ Types de données

PHP gère automatiquement les types, mais il est important de les connaître :

- **String** : "Bonjour"
- **Integer** : 42
- **Float** (nombres à virgule) : 15.5
- **Boolean** : true / false

9

Les variables

10

□ Affichage avec variables

```
<?php
$nom = "Dorra";
echo "Bonjour $nom";
?>
```

□ Concaténation

- Le point `.` est l'opérateur de **concaténation** en PHP.

```
<?php
$nom = "Dorra";
echo "Bonjour " . $nom;
?>
```

```
<?php
$nom_cours = "Programmation Web"; // Une chaîne de caractères (String)
$promotion = 2;                    // Un entier (Integer)
$est_difficile = false;           // Un booléen (Boolean)

echo "Bienvenue en " . $promotion . "ème année de " . $nom_cours;
?>
```

10

Les variables

11

Exemple Pratique : Mélanger HTML et PHP

- L'un des grands avantages du PHP est qu'il peut être injecté directement au milieu du code HTML.

The screenshot shows a code editor on the left with a PHP script embedded in an HTML document. The script calculates the current date and outputs it. On the right, a web browser window displays the rendered output of the script.

```

1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <title>Ma page PHP</title>
5 </head>
6 <body>
7   <?php
8     $date = date("d/m/Y");
9     echo "<h1>Nous sommes le $date</h1>";
10   <?>
11   <p>Ce paragraphe est écrit en HTML pur.</p>
12 </body>
13 </html>

```

The browser window shows the output: **Nous sommes le 15/04/2026** and the paragraph "Ce paragraphe est écrit en HTML pur."

11

Les variables

12

Applications :

1. Écrire un script PHP qui déclare 3 variables non, prenom et age, puis il les affiche.
2. Écrire un script PHP qui déclare 2 variables : a et b, puis il affiche leur somme et leur multiplication.

```

<?php
$nom = "VotreNom";
$prenom = "VotrePrenom";
$age = 22;

echo "Nom : " . $nom . "<br>";
echo "Prénom : " . $prenom . "<br>";
echo "Age : " . $age;
?>

```

```

<?php
$a = 10;
$b = 5;

$somme = $a + $b;
$produit = $a * $b;

echo "La somme de $a et $b est : " . $somme . "<br>";
echo "La multiplication de $a et $b est : " . $produit;
?>

```

12

13

Intégration du PHP

13

Intégration du PHP

14

- PHP est un langage côté serveur
- Il s'intègre directement dans les fichiers .php
- Il permet de générer du HTML dynamiquement.
- On peut intégrer PHP dans un document HTML de plusieurs façons :
 1. Dans le corps (body) du document
 2. Dans l'entête (head)
 3. Via un fichier externe
 4. Dans le HTML (attributs dynamiques)
 5. Génération complète de HTML

14

Intégration du PHP

15

Méthode 1 : Dans le corps (**body**)

- C'est la méthode la plus utilisée
- Permet d'afficher du contenu dynamique
- Simple et lisible

```
<!DOCTYPE html>
<html>
<head>
  <title>Mon premier PHP</title>
</head>
<body>

  <h1>
  <?php
  echo "Bienvenue en PHP !";
  ?>
  </h1>

</body>
</html>
```

15

Intégration du PHP

16

Méthode 2 : Dans l'entête du document (**<head>**)

- Utilisation : Déclaration de variables

Préparation des données

→ Utile pour configurer la page.

```
<!DOCTYPE html>
<html>
<head>
  <?php
  $titre = "Ma page PHP";
  ?>
  <title><?php echo $titre; ?></title>
</head>

<body>
  <h1>Bienvenue</h1>
</body>
</html>
```

16

Intégration du PHP

17

Méthode 3 : Dans un fichier externe PHP

- Organisation et Réutilisation du code.

```
// fichier fonctions.php
<?php
function bonjour() {
    return "Bonjour";
}
?>
```

```
// fichier principal
<?php
include "fonctions.php";
echo bonjour();
?>
```

```
<? PHP
<!DOCTYPE html>
<html>
<body>

<?php
include "fonctions.php";
?>

<h1>
<?php echo bonjour(); ?>
</h1>

</body>
</html>
```

17

Intégration du PHP

18

Méthode 4 : Dans les attributs HTML (cas dynamique)

- Permet de modifier le style ou contenu

```
<?php
$couleur = "red";
?>

<p style="color: <?php echo $couleur; ?>">
  Texte en couleur
</p>
```

18

Intégration du PHP

19

Méthode 5 : Génération complète de HTML

- PHP peut générer toute la page.
- Puissant mais moins lisible → à éviter en excès.

```
<? PHP  
  
<?php  
echo "<h1>Bienvenue en PHP</h1>";  
echo "<p>Ceci est généré dynamiquement</p>";  
?>
```

19

20

Structures de contrôle

20

Structures de contrôle conditionnelles

21

En PHP, les structures conditionnelles sont très proches de celles du JavaScript ou du C.

- A. **Le if, elseif, else** : C'est la structure la plus classique. On utilise des parenthèses pour la condition et des accolades pour le bloc d'instructions.

```
<?php
$age = 20;

if ($age >= 18) {
    echo "Majeur";
}
?>
```

```
<?php
$age = 15;

if ($age >= 18) {
    echo "Majeur";
} else {
    echo "Mineur";
}
?>
```

```
<?php
$note = 14;

if ($note >= 16) {
    echo "Très bien";
} elseif ($note >= 10) {
    echo "Passable";
} else {
    echo "Échec";
}
?>
```

21

Structures de contrôle conditionnelles

22

- B. **Le switch** : Idéal lorsqu'on doit comparer une même variable à plusieurs valeurs différentes.

```
<?php
$jour = 3;

switch ($jour) {
    case 1:
        echo "Lundi";
        break;
    case 2:
        echo "Mardi";
        break;
    case 3:
        echo "Mercredi";
        break;
    default:
        echo "Jour invalide";
}
?>
```

```
<?php
$note = 18;

switch ($note) {
    case 20:
        echo "Parfait !";
        break;
    case 18:
    case 19:
        echo "Excellent";
        break;
    default:
        echo "Bon travail";
}
?>
```

22

Structures de contrôle itératives

Les boucles

23

- Les boucles permettent de répéter un bloc de code plusieurs fois.

1. La boucle **while** (Tant que) :

```
<?php
$i = 1;

while ($i <= 5) {
    echo $i . "<br>";
    $i++;
}
?>
```

2. La boucle **do ... while** (Répéter jusqu'à) :

```
<?php
$i = 1;

do {
    echo $i . "<br>";
    $i++;
} while ($i <= 5);
?>
```

23

Structures de contrôle itératives

Les boucles

24

- Les boucles permettent de répéter un bloc de code plusieurs fois.

3. La boucle **for** (Pour) :

```
<?php
// for (initialisation ; condition ; incrémentation)
for ($i = 0; $i < 10; $i++) {
    echo "Le carré de $i est " . ($i * $i) . "<br>";
}
?>
```

```
<?php
for ($i = 1; $i <= 5; $i++) {
    echo $i . "<br>";
}
?>
```

4. La boucle **foreach** (Spécifique aux tableaux) :

```
<?php
$etudiants = ["Amine", "Sarrah", "Yassine"];

foreach ($etudiants as $nom) {
    echo "Étudiant : $nom <br>";
}
?>
```

```
<?php
$tab = [10, 20, 30];

foreach ($tab as $valeur) {
    echo $valeur . "<br>";
}
?>
```

24

Structures de contrôle itératives

Les boucles

25

4. La boucle **foreach** (Spécifique aux tableaux) :

```
<?php
$etudiants = ["Amine", "Sarrah", "Yassine"];

foreach ($etudiants as $nom) {
    echo "Étudiant : $nom <br>";
}
?>
```

- **syntaxe alternative (très propre pour le HTML)** : il existe une syntaxe plus lisible pour intégrer des boucles dans du HTML lourd :

- On remplace { par : et } par **endforeach**; ou **endif**;

```
<ul>
    <?php foreach ($etudiants as $nom): ?>
        <li><?php echo $nom; ?></li>
    <?php endforeach; ?>
</ul>
```

25

La boucle foreach — Valeur vs Référence

26

1. Le comportement par défaut : Passage par VALEUR

- Par défaut, PHP crée une **copie** de la valeur de l'élément du tableau dans la variable de boucle. Modifier cette variable ne change pas le tableau original.

```
$notes = [10, 12, 14];

foreach ($notes as $n) {
    $n = $n + 2; // On modifie la COPIE ($n)
}

print_r($notes);
// Résultat : [10, 12, 14] -> Le tableau n'a PAS changé.
```

```
$prix = [100, 200, 300];

foreach ($prix as $p) {
    $p = $p * 0.8; // On réduit de 20%
}

echo $prix[0]; // Affiche encore 100 !
```

26

La boucle foreach — Valeur vs Référence

27

2. Le passage par RÉFÉRENCE (&)

- Si l'on souhaite modifier directement les éléments du tableau original, on utilise le symbole **&** devant la variable de boucle.
- La variable devient alors un "alias" de la case mémoire du tableau.

```
$notes = [10, 12, 14];

foreach ($notes as &$n) { // Notez le &
    $n = $n + 2; // On modifie l'ORIGINAL
}

print_r($notes);
// Résultat : [12, 14, 16] -> Le tableau est mis à jour !
```

27

La boucle foreach — Valeur vs Référence

28

Attention : Après une boucle par référence, la variable `$n` reste liée au dernier élément du tableau.

Si on réutilise `$n` plus tard, on risque de modifier le dernier élément par erreur.

→ Toujours casser la référence après la boucle avec **`unset()`**.

```
$prix = [100, 200, 300];

foreach ($prix as &$p) { // On pointe vers l'original
    $p = $p * 0.8;
}

unset($p); // Bonne pratique : on libère la référence

echo $prix[0]; // Affiche 80 !
```

28

Exemple Pratique

29

□ Générer un tableau HTML dynamiquement

```
<table border="1">
  <tr>
    <th>Multiplicateur</th>
    <th>Résultat (x5)</th>
  </tr>
  <?php
  for ($i = 1; $i <= 10; $i++) {
    echo "<tr>";
    echo "<td> $i </td>";
    echo "<td>" . ($i * 5) . "</td>";
    echo "</tr>";
  }
  ?>
</table>
```

Multiplicateur	Résultat (x5)
1	5
2	10
3	15
4	20
5	25
6	30
7	35
8	40
9	45
10	50

29

30

Les tableaux

30

Définition

31

- Un tableau en PHP est une structure de données qui permet de stocker **plusieurs valeurs dans une seule variable**.

Exemple : liste d'étudiants, notes, produits, ...

- En PHP, un tableau n'est pas seulement une liste ; c'est une structure qui peut associer des clés à des valeurs.
- On distingue deux types principaux :
 - A. Les Tableaux Indexés
 - B. Les Tableaux associatifs

31

Les Tableaux Indexés

32

- C'est le tableau classique où chaque élément est associé à un indice numérique commençant par **0**.

```
<?php
// Déclaration
$matieres = array("PHP", "Réseaux", "Base de données");
// Syntaxe courte (plus moderne)
$langages = ["HTML", "CSS", "JavaScript"];

// Accès
echo $matieres[0]; // Affiche : PHP

// Ajouter un élément à la fin
$matieres[] = "Algorithmique";
?>
```

```
<?php
$notes = [12, 15, 10, 18];

for ($i = 0; $i < count($notes); $i++) {
    echo $notes[$i] . "<br>";
}
?>
```

32

Les tableaux associatifs

33

- C'est ici que PHP devient très puissant. Au lieu d'utiliser des chiffres (0, 1, 2), on utilise des **clés personnalisées** (chaînes de caractères). C'est l'équivalent des dictionnaires en Python.

```
<?php
// Définition d'un tableau associatif représentant un étudiant type
$etudiant = [
    "nom" => "Ben Foulen",
    "prenom" => "Foulen",
    "classe" => "L2 Info",
    "moyenne" => 15.5
];

// Accès aux données via les clés personnalisées
echo "L'étudiant " . $etudiant["prenom"] . " " . $etudiant["nom"] .
    " est inscrit en " . $etudiant["classe"] . ".";

// Exemple de modification de la moyenne
$etudiant["moyenne"] = 16.0;
?>
```

```
<?php
$etudiant = [
    "nom" => "Ali",
    "age" => 20,
    "note" => 16
];

echo $etudiant["nom"];
?>
```

33

Les tableaux associatifs

34

- **Parcourir un tableau avec foreach** : C'est la méthode la plus recommandée. Elle évite de gérer manuellement l'indice \$i.

```
<?php
// Pour un tableau associatif, on récupère la clé ET la valeur
foreach ($etudiant as $cle => $valeur) {
    echo "$cle : $valeur <br>";
}
?>
```

```
<?php
$etudiant = [
    "nom" => "Ali",
    "age" => 20,
    "note" => 16
];

foreach ($etudiant as $cle => $valeur) {
    echo $cle . " : " . $valeur . "<br>";
}
?>
```

34

35

Les Fonctions en PHP

35

Déclaration d'une fonction

36

- Une fonction est un bloc de code réutilisable qui exécute une tâche précise.
- Les fonctions permettent de modulariser le code, d'éviter les répétitions et de rendre le script plus lisible.
- La syntaxe est simple : on utilise le mot-clé **function**.

```
<?php
function direBonjour() {
    echo "Bonjour les étudiants";
}

direBonjour();
?>
```

36

Fonction avec paramètres

37

□ Sans retour

```
<?php
function direBonjour($nom) {
    echo "Bonjour " . $nom;
}

direBonjour("Ali");
?>
```

□ Avec retour (return)

```
<?php
function saluer($nom) {
    return "Bonjour " . $nom . " !";
}

echo saluer("les étudiants");
?>
```

37

Fonction avec paramètres

38

□ avec plusieurs paramètres

```
<?php
function addition($a, $b) {
    return $a + $b;
}

$resultat = addition(5, 3);
echo $resultat;
?>
```

```
<?php
function calculer($a, $b) {
    return $a * $b;
}

echo calculer(4, 6);
?>
```

□ Paramètres avec valeurs par défaut

```
<?php
function calculerPrix($prixHT, $tva = 0.19) { // 19% par défaut (Tunisie)
    $prixTTC = $prixHT * (1 + $tva);
    return $prixTTC;
}

echo calculerPrix(100); // Utilise 0.19
echo calculerPrix(100, 0.07); // Utilise 0.07
?>
```

38

La portée des variables (Variable Scope)

39

- **Attention** : C'est un point de confusion fréquent. En PHP, une variable définie à l'extérieur d'une fonction n'est pas accessible à l'intérieur de celle-ci (contrairement au JavaScript).

```
<?php
$ecole = "ISIMM";

function afficherEcole() {
    // echo $ecole; // ERREUR !
    global $ecole; // On doit préciser qu'on utilise la variable globale
    echo $ecole;
}
?>
```

39

Fonctions + tableaux + HTML

40

Exemple : Transformer un tableau associatif en une liste HTML

```
<?php
$infos = ["FST", "ENISo", "ISIMM"];

function genererListe($tab) {
    echo "<ul>";
    foreach ($tab as $element) {
        echo "<li>$element</li>";
    }
    echo "</ul>";
}

// Appel de la fonction dans la page
genererListe($infos);
?>
```

40

41

Gestion des Formulaires

(Méthodes GET / POST)

41

Gestion des Formulaires (GET & POST)

42

- Pour qu'un serveur PHP puisse traiter des données, elles doivent lui être envoyées par un client (le navigateur) via une requête HTTP.
- PHP utilise deux tableaux spéciaux, appelés **superglobales**, pour récupérer ces données : **\$_GET** et **\$_POST**.

Structure d'un formulaire HTML :

```
<form method="post" action="traitement.php">
  Nom : <input type="text" name="nom"><br>
  <input type="submit" value="Envoyer">
</form>
```

method → méthode d'envoi (GET ou POST)

action → fichier PHP qui traite les données

name → identifiant de la donnée

42

Méthode GET

43

- Les données sont transmises directement dans l'URL de la page.

Syntaxe URL : recherche.php?cle=valeur&autre_cle=autre_valeur

Utilisation : Pour des actions qui ne modifient pas les données (recherche, tri, pagination).

Limites : Les données sont visibles par tous, limitées en taille (environ 2000 caractères) et ne permettent pas l'envoi de fichiers.

```
<form method="get" action="traitement.php">
  Nom : <input type="text" name="nom">
  <input type="submit">
</form>
```

URL obtenue : <http://localhost/traitement.php?nom=Ali>

Récupération en PHP :

```
<?php
echo $_GET["nom"];
?>
```

43

Méthode GET

44

- Exemple de recherche :

HTML

```
<form action="traitement.php" method="GET">
  <label>Votre recherche :</label>
  <input type="text" name="mot_cle">
  <button type="submit">Rechercher</button>
</form>
```

```
<?php
// traitement.php
$recherche = $_GET['mot_cle'];
echo "Vous avez recherché : " . $recherche;
?>
```

44

Méthode POST

45

- Les données sont envoyées de manière **"invisible"** dans le corps de la requête HTTP.
- **Utilisation** : Pour des actions qui modifient des données sur le serveur (formulaire d'inscription, connexion, ajout d'un article).
- **Avantages** : Plus sécurisé pour les mots de passe (non visible dans l'URL), pas de limite de taille, permet l'envoi de fichiers (images, PDF).

```
<form action="save.php" method="POST">
  <input type="text" name="login" placeholder="Identifiant">
  <input type="password" name="pass" placeholder="Mot de passe">
  <button type="submit">S'inscrire</button>
</form>
```

```
<?php
// save.php
$user = $_POST['login'];
$password = $_POST['pass'];
echo "Compte créé pour : " . $user;
?>
```

45

Vérifier l'existence des données : **isset()**

46

- Pour éviter les erreurs *"Notice: Undefined index"*, il faut toujours vérifier si la donnée a bien été envoyée avec la fonction **isset()**.
- Si on essaie d'accéder à `$_POST['login']` alors que le formulaire n'a pas été soumis, PHP affichera une erreur. Il faut toujours tester :

```
<?php
if (isset($_POST['login'])) {
    // On traite la donnée
}
?>
```

- **Vérifier si vide** : avec la fonction **empty()**

```
<?php
if (empty($_POST['login'])) {
    echo "Champ obligatoire";
}
?>
```

46

Nettoyer les données : htmlspecialchars()

47

- Pour éviter les attaques **XSS** (un utilisateur qui injecterait du JavaScript malveillant dans un champ texte), on utilise cette fonction qui transforme les caractères spéciaux en entités HTML.
- La fonction **htmlspecialchars()** est indispensable pour se protéger contre les failles **XSS** en neutralisant les balises HTML saisies par un utilisateur malveillant.
- Elle transforme les caractères comme < ou > en entités HTML (<, >), empêchant l'exécution de scripts malveillants.

```
<?php
// Sécurité minimum
$nom = htmlspecialchars($_POST['nom']);
?>
```

```
<?php
if (isset($_POST['nom_utilisateur'])) {
    $nom = $_POST['nom_utilisateur'];
    echo "Bienvenue, " . htmlspecialchars($nom);
}
?>
```

47

Exercice : calculatrice simple

48

- Créer une page qui demande deux nombres et une opération, puis affiche le résultat :
 - Un formulaire avec deux champs numériques et une liste déroulante pour choisir l'opération (+, -, *, /).
 - Le script PHP devra récupérer les données et afficher le résultat.

```
<form method="POST">
    <input type="number" name="n1" required>
    <select name="op">
        <option value="+">+</option>
        <option value="-">-</option>
        <option value="*">*</option>
    </select>
    <input type="number" name="n2" required>
    <button type="submit">Calculer</button>
</form>
```

48

Exercice : calculatrice simple

49

- Créer une page qui demande deux nombres et une opération, puis affiche le résultat :
 - Un formulaire avec deux champs numériques et une liste déroulante pour choisir l'opération (+, -, *, /).
 - Le script PHP devra récupérer les données et afficher le résultat.

```
<?php
if (isset($_POST['n1'], $_POST['n2'], $_POST['op'])) {
    $v1 = $_POST['n1'];
    $v2 = $_POST['n2'];
    $op = $_POST['op'];

    if ($op == "+") $res = $v1 + $v2;
    elseif ($op == "-") $res = $v1 - $v2;
    elseif ($op == "*") $res = $v1 * $v2;

    echo "<h3>Résultat : $res</h3>";
}
?>
```

49

50

Sessions et Cookies

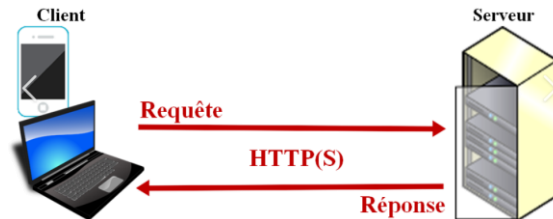
La Gestion de l'État

50

Sessions et Cookies

51

- Par défaut, le protocole HTTP est "stateless" (sans mémoire).
- Cela signifie que le serveur oublie qui vous êtes dès qu'une nouvelle page est chargée.



- Le protocole HTTP est dit "sans état" car chaque transaction (Requête/Réponse) est indépendante.
- Par le simple biais du protocole, le serveur ne "se souvient" pas de la requête précédente.

51

Sessions et Cookies

52

- Pourtant, nos usages modernes exigent de la continuité.
- Pour répondre aux besoins d'applications de plus en plus complexes (e-commerce, réseaux sociaux), nous devons maintenir un **état artificiel**.
- C'est là qu'interviennent les **sessions**.
- En utilisant des cookies ou des headers, nous permettons au serveur de reconnaître un utilisateur au fil de sa navigation, recréant ainsi une illusion de persistance sur un protocole qui, par nature, oublie tout instantanément.

52

Les Sessions (Côté Serveur)

53

- La session permet de stocker des informations sur le serveur pour un utilisateur spécifique. Ces données sont disponibles sur toutes les pages du site tant que le navigateur n'est pas fermé.
- **Utilisation** : Stocker l'identifiant d'un utilisateur connecté, le contenu d'un panier d'achat.
- **Fonctionnement** :
 - **session_start()** : Cette fonction doit être appelée au tout début de **chaque** fichier PHP (avant tout code HTML) pour démarrer ou récupérer une session → **Obligatoire** avant toute utilisation.
 - **\$_SESSION** : C'est la superglobale (tableau associatif) où l'on stocke les données.

53

Les Sessions (Côté Serveur)

54

- Une session stocke des données **côté serveur**.

```
<?php
// page1.php
session_start();          // Démarrer une session
$_SESSION['user'] = "Foulen";    // Stocker des données
$_SESSION['role'] = "Etudiant";
echo "Session démarrée pour " . $_SESSION['user']; // Récupérer
?>

<?php
// page2.php
session_start();
if(isset($_SESSION['user'])) {
    echo "Bonjour " . $_SESSION['user'] . ", vous êtes toujours connecté.";
}
?>
```

- **Détruire une session :**

```
<?php
session_destroy();
?>
```

54

Les Sessions (Côté Serveur)

55

□ **Exemple** : Login simple

```
<?php
session_start();

$login = "admin";
$password = "1234";

if ($_POST["login"] == $login && $_POST["password"] == $password) {
    $_SESSION["user"] = $login;
    echo "Bienvenue " . $_SESSION["user"];
} else {
    echo "Erreur";
}
?>
```

55

Les Cookies (Côté Client)

56

- Un cookie est une donnée stockée **dans le navigateur**.
- Contrairement à la session, il peut rester même après la fermeture du navigateur.

Utilisation : Retenir la langue préférée, "se souvenir de moi".

```
<?php
// Créer un cookie qui dure 1 jour (86400 secondes)
setcookie("theme", "sombre", time() + 86400);

// Récupérer un cookie
if(isset($_COOKIE['theme'])) {
    echo "Votre thème est : " . $_COOKIE['theme'];
}
?>
```

56

Les Cookies (Côté Client)

57

- Un cookie est une donnée stockée **dans le navigateur**.

- **Création d'un cookie :**

```
<?php
setcookie("nom", "Ali", time() + 3600);
?>
```

- **Récupération :**

```
<?php
echo $_COOKIE["nom"];
?>
```

- **Suppression :**

```
<?php
setcookie("nom", "", time() - 3600);
?>
```

57

Session vs Cookie

58

Critère	Session	Cookie
Stockage	serveur	navigateur
Sécurité	✓ plus sécurisé	✗ moins sécurisé
Durée	temporaire	configurable

- **Session** = authentification / données sensibles
- **Cookie** = préférences utilisateur (langue, thème)

58